TruthTrack Deployment Guide

Complete guide for deploying TruthTrack to production on iOS and Android.

Pre-Deployment Checklist

- [ ] All environment variables configured
- [ ] Database backed up
- [ ] API endpoints tested
- [ ] Frontend tested on iOS and Android simulators
- [ ] SSL certificates obtained
- [ ] Rate limiting configured
- [ ] Monitoring and logging set up
- [ ] Privacy policy and terms of service finalized

Backend Deployment

Option 1: Heroku

Bash

```bash
# Install Heroku CLI
npm install -g heroku

# Login to Heroku
heroku login

# Create app
heroku create truthtrack-backend

# Set environment variables
heroku config:set DATABASE_URL=your-database-url
heroku config:set JWT_SECRET=your-secret
heroku config:set OPENAI_API_KEY=your-key
heroku config:set USDA_API_KEY=your-key
heroku config:set NUTRITIONIX_API_ID=your-id
heroku config:set NUTRITIONIX_API_KEY=your-key
```

```
# Deploy
git push heroku main
```

Option 2: AWS EC2

Bash

```bash
# Launch EC2 instance
# SSH into instance
ssh -i key.pem ec2-user@your-instance

# Install Node.js
curl -sL https://rpm.nodesource.com/setup_18.x | sudo bash -
sudo yum install nodejs

# Clone repository
git clone your-repo
cd truthtrack-mobile/backend

# Install dependencies
npm install

# Set environment variables
export DATABASE_URL=your-database-url
export JWT_SECRET=your-secret
# ... other variables

# Install PM2 for process management
npm install -g pm2

# Start server
pm2 start npm --name "truthtrack" -- start

# Set up SSL with Let's Encrypt
sudo yum install certbot
sudo certbot certonly --standalone -d your-domain.com
```

Option 3: DigitalOcean App Platform

Bash

```bash
# Push to GitHub
git push origin main

# Connect DigitalOcean to GitHub
# Create new app from GitHub repo
```

```
# Configure environment variables in DigitalOcean dashboard
# Deploy
```

Database Setup

MySQL on AWS RDS

Bash

```bash
# Create RDS instance
aws rds create-db-instance \
  --db-instance-identifier truthtrack-db \
  --db-instance-class db.t3.micro \
  --engine mysql \
  --master-username admin \
  --master-user-password your-password \
  --allocated-storage 20

# Get endpoint
aws rds describe-db-instances --db-instance-identifier truthtrack-db

# Connect and create database
mysql -h your-endpoint.rds.amazonaws.com -u admin -p
CREATE DATABASE truthtrack;

# Run migrations
npm run db:migrate
```

TiDB Cloud

Bash

```bash
# Create TiDB cluster on TiDB Cloud console
# Get connection string
# Update DATABASE_URL

# Run migrations
npm run db:migrate
```

Frontend Deployment

iOS

Prerequisites

- Apple Developer Account ($99/year)
- Mac with Xcode
- iOS device or simulator

Build and Submit

Bash

```bash
# Install EAS CLI
npm install -g eas-cli

# Login to Expo
eas login

# Configure build
eas build:configure --platform ios

# Build for iOS
eas build --platform ios --auto-submit

# Follow prompts to:
# - Select provisioning profile
# - Sign with certificate
# - Submit to App Store
```

Manual Submission

Bash

```bash
# Build locally
eas build --platform ios --local

# Open in Xcode
open ios/truthtrack.xcworkspace

# Archive and submit to App Store
# Product → Archive → Distribute App
```

Android

Prerequisites

- Google Play Developer Account ($25 one-time)
- Android SDK
- Keystore file

Build and Submit

Bash

```bash
# Generate keystore (first time only)
keytool -genkey-pair -v -keystore truthtrack.keystore \
  -keyalg RSA -keysize 2048 -validity 10000 \
  -alias truthtrack-key

# Configure build
eas build:configure --platform android

# Build for Android
eas build --platform android --auto-submit

# Follow prompts to:
# - Select keystore
# - Configure signing
# - Submit to Google Play
```

Manual Submission

Bash

```bash
# Build APK
eas build --platform android --local

# Upload to Google Play Console
# Release → Create new release → Upload APK
```

Environment Configuration

Production .env

Plain Text

```
# Database
DATABASE_URL=mysql://user:password@prod-db-host:3306/truthtrack
```

```
# Security
JWT_SECRET=use-strong-random-secret-here-min-32-chars
NODE_ENV=production

# APIs
OPENAI_API_KEY=sk-prod-key-here
USDA_API_KEY=prod-key-here
NUTRITIONIX_API_ID=prod-id-here
NUTRITIONIX_API_KEY=prod-key-here

# Server
PORT=3001
CORS_ORIGIN=https://your-domain.com,https://app.your-domain.com

# File Upload
MAX_FILE_SIZE=10485760
UPLOAD_DIR=/var/uploads
```

Frontend Configuration

Update `app.json`:

JSON

```
{
  "expo": {
    "name": "TruthTrack",
    "slug": "truthtrack",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "splash": {
      "image": "./assets/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    },
    "ios": {
      "supportsTabletMode": true,
      "bundleIdentifier": "com.truthtrack.app"
    },
    "android": {
      "adaptiveIcon": {
        "foregroundImage": "./assets/adaptive-icon.png",
        "backgroundColor": "#ffffff"
      },
      "package": "com.truthtrack.app"
    },
```

```
    "extra": {
      "apiUrl": "https://api.your-domain.com"
    }
  }
}
```

SSL/TLS Configuration

Using Let's Encrypt with Nginx

Bash

```bash
# Install Nginx
sudo apt-get install nginx

# Install Certbot
sudo apt-get install certbot python3-certbot-nginx

# Get certificate
sudo certbot certonly --nginx -d your-domain.com

# Configure Nginx
sudo nano /etc/nginx/sites-available/default
```

Nginx config:

Plain Text

```
server {
    listen 443 ssl http2;
    server_name api.your-domain.com;

    ssl_certificate /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;

    location / {
        proxy_pass http://localhost:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

```
server {
    listen 80;
    server_name api.your-domain.com;
    return 301 https://$server_name$request_uri;
}
```

Monitoring & Logging

Application Monitoring

Bash

```bash
# Install PM2 monitoring
pm2 install pm2-auto-pull
pm2 install pm2-logrotate

# View logs
pm2 logs truthtrack

# Monitor performance
pm2 monit
```

Database Monitoring

Bash

```bash
# MySQL slow query log
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 2;

# Check database size
SELECT table_schema,
       ROUND(SUM(data_length + index_length ) / 1024 / 1024, 2) AS size_mb
FROM information_schema.tables
GROUP BY table_schema;
```

Logging with ELK Stack

Bash

```bash
# Install Elasticsearch, Logstash, Kibana
docker run -d --name elasticsearch docker.elastic.co/elasticsearch/elasticsearc
docker run -d --name kibana docker.elastic.co/kibana/kibana:8.0.0
```

```
# Configure app to send logs to Elasticsearch
npm install winston winston-elasticsearch
```

Backup & Recovery

Database Backups

Bash

```
# Automated daily backup
0 2 * * * mysqldump -u root -p$MYSQL_PASSWORD truthtrack > /backups/truthtrack-

# Restore from backup
mysql -u root -p truthtrack < /backups/truthtrack-20240101.sql
```

S3 Backups

Bash

```
# Install AWS CLI
pip install awscli

# Configure credentials
aws configure

# Upload backups to S3
aws s3 sync /backups s3://truthtrack-backups/
```

Performance Optimization

Database Optimization

SQL

```
-- Add indexes
CREATE INDEX idx_user_meals ON mealLogs(userId, loggedAt);
CREATE INDEX idx_food_name ON foodDatabase(foodName);
CREATE INDEX idx_barcode ON foodDatabase(barcode);

-- Analyze tables
```

```
ANALYZE TABLE mealLogs;
ANALYZE TABLE foodDatabase;
```

API Caching

Bash

```bash
# Install Redis
sudo apt-get install redis-server

# Configure app to use Redis
npm install redis ioredis
```

Image Optimization

Bash

```bash
# Compress images before upload
npm install sharp

# Implement in visionService.ts
const compressed = await sharp(imageBuffer)
  .resize(1920, 1440, { fit: 'inside', withoutEnlargement: true })
  .jpeg({ quality: 80 })
  .toBuffer();
```

Security Hardening

API Rate Limiting

Bash

```bash
npm install express-rate-limit

# Configure in backend/src/index.ts
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100 // limit each IP to 100 requests per windowMs
});
```

```
app.use('/api/', limiter);
```

CORS Configuration

TypeScript

```
app.use(cors({
  origin: process.env.CORS_ORIGIN.split(','),
  credentials: true,
  optionsSuccessStatus: 200
}));
```

Input Validation

Bash

```
npm install joi

# Validate all inputs
const schema = Joi.object({
  email: Joi.string().email().required(),
  password: Joi.string().min(8).required()
});
```

Monitoring Checklist

- [ ] Server uptime monitoring
- [ ] Database performance monitoring
- [ ] API response time monitoring
- [ ] Error rate monitoring
- [ ] User activity logging
- [ ] Security audit logging
- [ ] Backup verification
- [ ] SSL certificate expiration alerts

Rollback Procedure

If deployment fails:

```bash
# Revert to previous version
git revert HEAD
git push origin main

# Restart server
pm2 restart truthtrack

# Restore database from backup
mysql -u root -p truthtrack < /backups/truthtrack-previous.sql
```

Post-Deployment

1. Test all functionality in production
2. Monitor error logs for issues
3. Verify database backups are working
4. Set up monitoring alerts
5. Announce app availability
6. Gather user feedback
7. Plan for future updates

Support

For deployment issues, contact:

- Backend support: backend-team@truthtrack.com
- DevOps support: devops@truthtrack.com
- App store support: app-support@truthtrack.com